

SQL – RIPASSO DI INIZIO ANNO

Esercizi di interrogazione e gestione dei database – Classe quarta

Liceo Scientifico – Scienze Applicate

Nome e cognome: _____ **Classe:** _____

Come usare questa scheda: tutti gli esercizi si basano sull'unico database GESTIONE_SCUOLA, descritto e popolato nelle pagine seguenti. Scrivi le tue query nello spazio previsto e verificane il risultato confrontandolo, quando presente, con l'esempio proposto. Le soluzioni non sono incluse: confrontati con i compagni e con il/la docente per la correzione. Nelle query utilizza SQL standard; dove utile, il tipo di JOIN o la clausola da usare sono lasciati alla tua scelta progettuale.

IL DATABASE GESTIONE_SCUOLA

Tutti gli esercizi di questa scheda si basano su un unico database, denominato GESTIONE_SCUOLA, composto dalle tabelle descritte di seguito.

STUDENTI — anagrafica degli studenti. Chiave primaria: id_studente.

MATERIE — elenco delle materie scolastiche. Chiave primaria: id_materia.

DOCENTI — anagrafica dei docenti, con la materia insegnata indicata come testo. Chiave primaria: id_docente.

VOTI — registro dei voti. Chiave primaria: id_voto. id_studente è chiave esterna verso STUDENTI, id_materia è chiave esterna verso MATERIE.

CORSI — corsi extracurricolari attivati nell'anno. Chiave primaria: id_corso.

ISCRIZIONI — tabella di collegamento tra studenti e corsi: id_studente è chiave esterna verso STUDENTI, id_corso è chiave esterna verso CORSI. Uno studente può iscriversi a più corsi e un corso può avere più studenti iscritti.

Le tabelle CORSI e ISCRIZIONI sono utilizzate soltanto negli esercizi più avanzati (in particolare nel mini-progetto finale).

Dati di esempio

Le tabelle seguenti riportano i dati con cui puoi verificare le tue query.

STUDENTI

id_studente	nome	cognome	classe	data_nascita
1	Alice	Ferrari	4A	2007-03-12
2	Luca	Moretti	4A	2007-07-25
3	Sara	Greco	4A	2007-01-30
4	Davide	Conti	4B	2007-05-18
5	Giorgia	Fontana	4B	2007-09-02
6	Marco	Villa	4B	2007-11-14
7	Elisa	Marino	4A	2007-04-08
8	Tommaso	Rizzo	4B	2007-02-21

MATERIE

id_materia	nome_materia
1	Matematica
2	Italiano
3	Informatica
4	Inglese
5	Scienze

DOCENTI

id_docente	nome	cognome	materia
1	Laura	Bianchi	Matematica
2	Marco	Rossi	Italiano
3	Elena	Verdi	Informatica
4	Paolo	Neri	Inglese
5	Giulia	Colombo	Scienze

VOTI

id_voto	id_studente	id_materia	voto	data_voto
1	1	1	8	2025-10-05
2	1	2	6	2025-10-10
3	1	3	9	2025-10-12
4	2	1	5	2025-10-05
5	2	2	7	2025-10-10
6	2	4	6	2025-10-15
7	3	1	9	2025-10-05
8	3	3	8	2025-10-12
9	3	5	7	2025-10-18
10	4	1	6	2025-10-05
11	4	2	5	2025-10-10
12	4	4	8	2025-10-15
13	5	1	7	2025-10-05
14	5	3	9	2025-10-12
15	5	5	6	2025-10-18
16	6	1	4	2025-10-05
17	6	2	6	2025-10-10
18	6	4	5	2025-10-15
19	7	3	10	2025-10-12
20	7	5	8	2025-10-18

CORSI

id_corso	nome_corso	anno
1	Robotica	2025
2	Debate	2025
3	Certificazione Cambridge	2025

ISCRIZIONI

id_studente	id_corso
1	1
2	1
3	2
4	3
5	1
6	2
7	3

ESERCIZIO 1 – SELECT DI BASE

Difficoltà: ★☆☆

Obiettivo:

Acquisire dimestichezza con la sintassi di base dell'istruzione SELECT e con la proiezione di colonne specifiche.

Prerequisiti:

Struttura di una tabella, istruzione SELECT, clausola FROM, clausola WHERE.

Scenario:

La segreteria della scuola deve consultare rapidamente l'anagrafica degli studenti registrata nella tabella STUDENTI.

Consegna:

1. Visualizza tutte le colonne di tutti gli studenti presenti nella tabella STUDENTI.
2. Visualizza soltanto il nome e il cognome di tutti gli studenti.
3. Visualizza nome, cognome e classe degli studenti che frequentano la classe 4A.

Vincoli:

- utilizzare SELECT e FROM in tutte le query;
- per il punto 3, utilizzare la clausola WHERE.

Esempio:

Risultato atteso per la richiesta 3:

nome	cognome	classe
Alice	Ferrari	4A
Luca	Moretti	4A
Sara	Greco	4A
Elisa	Marino	4A

Spazio per lo svolgimento:

[illegible]

[illegible]

Difficoltà: ★☆☆

Ordinare i risultati di una query secondo uno o più criteri ed eliminare le righe duplicate.

ORDER BY, clausole ASC e DESC, DISTINCT.

In vista degli scrutini, la scuola vuole presentare elenchi ordinati e senza ripetizioni.

8. Visualizza tutti gli studenti ordinati per cognome, in ordine alfabetico crescente.
9. Visualizza tutti i voti della tabella VOTI ordinati dal più alto al più basso.
10. Visualizza l'elenco delle classi presenti nella tabella STUDENTI, senza ripetizioni.
11. Visualizza gli studenti della classe 4B ordinati per data di nascita (scegli tu se dal più giovane al più anziano o viceversa, indicandolo chiaramente).

- utilizzare ORDER BY con ASC oppure DESC in ogni query che lo richiede;
- utilizzare DISTINCT per la richiesta 3.

Risultato atteso per la richiesta 3:

classe
4A
4B

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Spazio per lo svolgimento:

Spazio per lo svolgimento:

ESERCIZIO 6 – HAVING

Difficoltà: ★★☆☆

Obiettivo:

Filtrare i gruppi ottenuti da un raggruppamento sulla base di una condizione riferita a un valore aggregato.

Prerequisiti:

GROUP BY, HAVING, funzioni di aggregazione, differenza tra WHERE e HAVING.

Scenario:

La scuola vuole individuare le materie e gli studenti che si distinguono per rendimento o per numerosità dei voti registrati.

Consegna:

19. Individua le materie (id_materia) la cui media dei voti è superiore a 7.
20. Individua gli studenti (id_studente) che hanno ricevuto almeno 3 voti.
21. Individua le materie che hanno ricevuto meno di 4 voti complessivi.

Vincoli:

- utilizzare GROUP BY insieme a HAVING in tutte le query;
- non utilizzare WHERE per filtrare condizioni riferite a valori aggregati: quel tipo di filtro va espresso con HAVING.

Spazio per lo svolgimento:

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

ESERCIZIO 7 – INNER JOIN

Difficoltà: ★★☆☆

Obiettivo:

Combinare dati provenienti da più tabelle collegate tra loro da chiavi primarie ed esterne.

Prerequisiti:

INNER JOIN, condizione ON, concetto di chiave primaria e chiave esterna, alias di tabella.

Scenario:

Ora che sai raggruppare i dati, il preside chiede un report leggibile, con i nomi al posto degli identificativi numerici.

Consegna:

22. Visualizza nome e cognome dello studente insieme al voto ottenuto, per tutti i voti registrati.
23. Visualizza nome dello studente, nome della materia e voto, per tutti i voti registrati.
24. Visualizza tutti i voti relativi alla materia "Informatica", con nome e cognome dello studente.
25. Visualizza nome e cognome degli studenti che hanno ottenuto almeno un voto pari a 9 oppure a 10.

Vincoli:

- utilizzare INNER JOIN con la relativa condizione ON;
- utilizzare alias per le tabelle, se ritieni che rendano la query più leggibile.

Esempio:

Risultato atteso per la richiesta 3 (parziale):

nome	cognome	voto
Alice	Ferrari	9
Sara	Greco	8
Giorgia	Fontana	9

Spazio per lo svolgimento:

[illegible]

Difficoltà: ★★ ★

Integrare JOIN, GROUP BY e funzioni di aggregazione per produrre report leggibili basati su più tabelle.

INNER JOIN, GROUP BY, funzioni di aggregazione, ORDER BY.

La scuola vuole un report leggibile, con i nomi al posto degli id, sul rendimento di studenti e materie.

26. Calcola la media dei voti di ciascuno studente, mostrando nome e cognome al posto dell'id.
27. Calcola la media dei voti di ciascuna materia, mostrando il nome della materia al posto dell'id.
28. Individua lo studente con la media più alta, mostrando nome, cognome e media.
29. Individua la materia che ha ricevuto il maggior numero di voti, mostrando il nome della materia e il conteggio.

- combinare JOIN e GROUP BY nella stessa query;
- per le richieste 3 e 4, ragiona su come individuare il valore "più alto" o "maggiore" tra i gruppi calcolati (può esserti utile un ordinamento).

[illegible]

ESERCIZIO 9 – SOTTOQUERY

Difficoltà: ★★ ★

Obiettivo:

Costruire query annidate, utilizzando il risultato di una query come termine di confronto per un'altra.

Prerequisiti:

Sottoquery, funzioni di aggregazione, JOIN, GROUP BY.

Scenario:

La scuola vuole individuare eccellenze e casi da monitorare confrontando ogni studente o materia con la media generale, senza calcolarla manualmente.

Consegna:

30. Individua gli studenti la cui media dei voti è superiore alla media generale di tutta la scuola.
31. Individua lo studente (o gli studenti) che ha ottenuto il voto più alto in assoluto tra tutti i voti registrati.
32. Individua le materie la cui media dei voti è superiore alla media complessiva calcolata su tutte le materie.

Vincoli:

- almeno una query deve contenere una sottoquery, nella clausola WHERE oppure nella clausola FROM;
- non è consentito calcolare "a mano" la media generale e riscriverla come numero fisso all'interno della query: deve essere la query stessa a calcolarla.

Spazio per lo svolgimento:

[illegible]

Difficoltà: ★★ ★

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

ERRORI DA EVITARE

Prima di considerare concluso un esercizio, ricontrolla la tua query rispetto a questi errori tipici.

- Confondere WHERE e HAVING: WHERE filtra le singole righe prima del raggruppamento, HAVING filtra i gruppi dopo l'aggregazione.
- Dimenticare la condizione ON in un JOIN, ottenendo combinazioni di righe prive di senso (prodotto cartesiano).
- Inserire nella SELECT colonne che non compaiono né nel GROUP BY né dentro una funzione di aggregazione.
- Confondere COUNT(*) — che conta tutte le righe — con COUNT(nome_colonna) — che ignora i valori NULL di quella colonna.
- Usare AND quando la condizione richiederebbe OR (o viceversa), ottenendo risultati vuoti o troppo ampi.
- Dimenticare ORDER BY quando la consegna richiede esplicitamente un ordinamento dei risultati.
- Confondere chiave primaria (identifica in modo univoco una riga della propria tabella) e chiave esterna (fa riferimento alla chiave primaria di un'altra tabella).
- Utilizzare una sottoquery quando basterebbe un semplice JOIN o una funzione di aggregazione, complicando inutilmente la query.
- Ottenere duplicati inattesi dopo un JOIN, quando una riga di una tabella si combina con più righe corrispondenti nell'altra.
- Usare INNER JOIN quando la consegna richiede di includere anche i casi senza corrispondenza (in quel caso serve un LEFT JOIN).

AUTOVALUTAZIONE

Al termine del lavoro, valuta con onestà il tuo livello di padronanza per ciascuna competenza, barrando la casella corrispondente.

Legenda — 1: Devo ripassare · 2: Riesco con qualche difficoltà · 3: Riesco autonomamente · 4: Riesco e so spiegare il procedimento

Competenza	1	2	3	4
So leggere lo schema di un database				
So individuare chiavi primarie e chiavi esterne				
So utilizzare SELECT				
So filtrare i dati con WHERE				
So ordinare i risultati				
So utilizzare DISTINCT				
So utilizzare le funzioni di aggregazione				
So utilizzare GROUP BY				
So utilizzare HAVING				
So effettuare JOIN tra tabelle				
So costruire interrogazioni su più tabelle				
So utilizzare una sottoquery				
So analizzare un problema e trasformarlo in una query SQL				

NOTE DEL DOCENTE